

Specification Elements

You are to create a C++ program using the following elements (in any order of your choosing):

- * These elements are to be conceptually related to a theme of your choosing. I do not want you to simply put the various task elements into a program in isolation. For example, you can make them all part of a product registration program. So, I expect to see all these elements related to an imaginary product registration form. This helps lower the plagiarism score. You can let the AI pick it, but I bet your plagiarism score will increase if you do.
- *
- * Create a Program Greeting section where you display your name and the unifying theme. Display the program greeting material at the very start of the program. Put any initialization code here as well.
- *
- * Create at least five objects. Two objects will have either an aggregation or composition relationship, and three objects will have a parent and two children relationship.
- * Put the objects with the inheritance relationships into a polymorphic data structure. I realize the only two objects which will work in this situation are the child objects, so don't worry about putting the ABC in the data structure. You can use your dynamic memory or you can use a vector. If you use a vector, you still need to use the dynamic memory for something else.
- * Make the parent an abstract base class.
- * Create at least one lambda or functor.
- * Add one convert constructor/operator to one object which can convert that object to and from a primitive type (int, float, string).
- * Overload the stream insertion and stream extraction operators for each of your objects.
- * Create a dynamic memory object (with helper functions as needed). Call this function when you need to grow or shrink your dynamic array. You can add other operations to this as well.
- * Prompts for program input. Re-prompt incorrect input until it is correct. I'll leave it up to you to decide what two errors you will check for, as this depends on the type of data you are reading in.
- * Display a summary of your dynamic memory in a Table with Borders.
- * Do not use global variables.
- * Do not use maps, stacks, or queues. However, you can build these structures on your own, you just can't use the STL.
- *
- * You may always add additional elements to your programs. However, if you do, it needs to be: 1) thematically correct (belong with your theme) and 2) demonstrate mastery of the material. Don't duplicate something already in the program (no Singleton again), but something new. It also needs to be from material covered in the last week of class.
- * See the syllabus for information on high plagiarism scores, submission instructions, and late policy.

Deliverable 1

- * Craft a prompt for ChatGPT (or your favorite LLM) to write an example program that meets the Specification Elements above.
- * You may need to play around with your GenAI prompt, so it produces running code.
- * It is your call if you want to fix any logical errors in the code. If you fix the AI code, you have less to write about. You can't find any subsequent errors if it doesn't run. I would try to comment out the broken code and keep running the program to find all the errors.

Homework 5
CISP 400
Spring 2025

- * You can prohibit the GenAI from using language features we haven't covered yet. You may want to do that because you can only use the Advanced Material evaluation dimension once, regardless of how often the GenAI does it. So, prohibiting it from using certain concepts should give you more to work with (if it listens to you!). For instance, if the GenAI code has objects and lambdas, you may want to prohibit objects and lambdas to give you more code to evaluate. Remember, code beyond what we covered in class can be useful to write about under Advanced Material (assuming the next code iteration uses them again). If you need to exclude some concepts, write it up because that is a deviation from the assignment. If you are forced to do this, I suggest coding it under the dimension of Algorithm and Efficiency.
- * Ensure it uses the conceptual theme you selected above, or you may use the GenAI-created theme.
- * Put a source file header on this program, which looks like this:

```
// Author: ChatGPT
// For: Caleb Fowler (your name)
// Homework (number), Deliverable 1
```

- * Turn in this source file in Canvas.

Deliverable 2

- * Rewrite the program in Deliverable 1 and improve it. This is where you get practice surpassing the chatbot's average results.
- * We defined it better according to the attributes we discussed in class. For example, one dimension is correctness, and the evaluation criteria is that the program "produces logically correct output." If the LLM doesn't do that and your program does, then you have improved the LLM program.
- * I expect you to improve the LLM in at least **six** dimensions. The same dimension problem may appear in multiple places in your program. That's fine; you can change it and discuss it occurring in multiple locations in your write-up, but it only counts once.
- * You don't need to copy the GenAI program; you can write it from scratch.
- * Put a source file header on this program, which looks like this:

```
// Author: Caleb Fowler (your name)
// Improvements to ChatGPT
// Homework (number), Deliverable 2
```

- * Turn in this source file in Canvas.

Deliverable 3

- * Write a report discussing:
 1. For each problem dimension, answer the following:
 1. List the problem dimension.
 1. **What** did you see with ChatGPT's code (what problem category did you find)? What did it do wrong?
 2. **Why** was that a problem, or what happened due to #1?
 3. **What** did you do to **correct** it in your version of the program?
 4. **What** was the result of your correction?
 2. What did the ChatBot do right? Why do you think it was a good decision?

Homework 5
CISP 400
Spring 2025

3. What did you learn from looking at the ChatBot code and improving on it? Do you think your programming skills improved as a result of this comparison?

- * I expect a minimum of one paragraph for each dimension you fixed and the other two topics.
- * This is a college essay using MLA research paper format.
- * Do not restate the deliverable text, assignment, or source code.
- * You CAN add one-line quotes as examples of the problem and solution. This is ok:

Line 10 `cout << "Integer: " << myInt << endl;`

I changed this I'm my code to:

Line 12 `cout << "Integer: " << myInt << "\n";`

- * Don't quote the syntax as you would in an essay; that messes up the C++ syntax.
- * It should be at least 500 words. I can assure you many students will write considerably more than this.
- * Turn this file in Canvas under deliverable 3.